1.Implementation of Symbol Table

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 100

typedef struct Symbol {
    char name[50];
    char type[20];
    int value;
    struct Symbol* next;
} Symbol;

typedef struct SymbolTable {
    Symbol* table[TABLE_SIZE];
} SymbolTable;

unsigned int hash(char* name) {
    unsigned int hash = 0;
    while (*name) {
        hash = (hash * 31) + *name++;
    }
    return hash % TABLE_SIZE;
}

SymbolTable* createSymbolTable() {
    SymbolTable* st = (SymbolTable*)malloc(sizeof(SymbolTable));
    for (int i = 0; i < TABLE_SIZE; i++) {
        st->table[i] = NULL;
    }
    return st;
}

void insert(SymbolTable* st, char* name, char* type, int value) {
    unsigned int index = hash(name);
    Symbol* newSymbol = (Symbol*)malloc(sizeof(Symbol));
    strcpy(newSymbol->name, name);
    strcpy(newSymbol->type, type);
    newSymbol->value = value;
    newSymbol->next = st->table[index];
    st->table[index] = newSymbol;
    printf("Inserted: %s\n", name);
}

Symbol* lookup(SymbolTable* st, char* name) {
    unsigned int index = hash(name);
    Symbol* current = st->table[index];
```

```
    while (current) {
        if (strcmp(current->name, name) == 0) {
            return current;
        }
        current = current->next;
    }
    return NULL;
}

void display(SymbolTable* st) {
    printf("\nSymbol Table:\n");
    printf("Name\t\tType\t\tValue\n");
    printf("-------------------------------\n");
    for (int i = 0; i < TABLE_SIZE; i++) {
        Symbol* current = st->table[i];
        while (current) {
            printf("%s\t\t%s\t\t%d\n", current->name, current->type, current->value);
            current = current->next;
        }
    }
}

int main() {
    SymbolTable* st = createSymbolTable();
    insert(st, "x", "int", 10);
    insert(st, "y", "float", 25);
    insert(st, "temp", "int", 100);
    insert(st, "result", "double", 45);

    display(st);

    Symbol* found = lookup(st, "x");
    if (found) {
        printf("\nFound variable 'x': Type=%s, Value=%d\n", found->type, found->value);
    }
    return 0;
}
```

Output:

```
Inserted: x
Inserted: y
Inserted: temp
Inserted: result

Symbol Table:
Name            Type            Value
-------------------------------
result          double          45
```

```
temp          int           100
x             int           10
y             float         25

Found variable 'x': Type=int, Value=10
```

7. Implement type checking.

typecheck.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef enum
{
        TYPE_INT,
        TYPE_FLOAT,
        TYPE_CHAR,
        TYPE_VOID,
        TYPE_ERROR
} DataType;
typedef struct TypeInfo
{
DataType type;
int isArray;
int arraySize;
} TypeInfo;
typedef struct Variable
{
char name[50];
TypeInfo typeInfo;
struct Variable* next;
} Variable;
Variable* symbolTable = NULL;
void addVariable(char* name, DataType type, int isArray, int size) {
Variable* var = (Variable*)malloc(sizeof(Variable));
strcpy(var->name, name);
var->typeInfo.type = type;
var->typeInfo.isArray = isArray;
var->typeInfo.arraySize = size;
var->next = symbolTable;
symbolTable = var;
char* typeStr = (type == TYPE_INT) ? "int" :
(type == TYPE_FLOAT) ? "float" :
(type == TYPE_CHAR) ? "char" : "void";
printf("Added variable %s: %s%s\n", name, typeStr, isArray ? " array" : "");
}
TypeInfo getVariableType(char* name) {
```

```
Variable* current = symbolTable;while (current) {
if (strcmp(current->name, name) == 0) {
return current->typeInfo;
}
current = current->next;
}
TypeInfo error = {TYPE_ERROR, 0, 0};
return error;
}
int areTypesCompatible(DataType type1, DataType type2) {
if (type1 == type2) return 1;
if ((type1 == TYPE_INT && type2 == TYPE_FLOAT) ||
(type1 == TYPE_FLOAT && type2 == TYPE_INT)) return 1;
return 0;
}
DataType checkBinaryOperation(char* var1, char* var2, char op) {
TypeInfo type1 = getVariableType(var1);
TypeInfo type2 = getVariableType(var2);
if (type1.type == TYPE_ERROR || type2.type == TYPE_ERROR) {
printf("Error: Undefined variable\n");
return TYPE_ERROR;
}
if (!areTypesCompatible(type1.type, type2.type)) {
printf("Error: Type mismatch in operation %s %c %s\n", var1, op, var2);
return TYPE_ERROR;
}
// Return the more general type
if (type1.type == TYPE_FLOAT || type2.type == TYPE_FLOAT) {
return TYPE_FLOAT;
}
return TYPE_INT;
}
int main() {
printf("Type Checking System\n");
printf("====================\n\n");
addVariable("x", TYPE_INT, 0, 0);
addVariable("y", TYPE_FLOAT, 0, 0);
addVariable("z", TYPE_CHAR, 0, 0);
addVariable("arr", TYPE_INT, 1, 10);
printf("\nType checking operations:\n");
DataType result = checkBinaryOperation("x", "y", '+');
if (result != TYPE_ERROR) {
printf("x + y: Valid operation, result type: %s\n",
result == TYPE_FLOAT ? "FLOAT" : "INT");
}
result = checkBinaryOperation("x", "z", '+');
if (result == TYPE_ERROR) {
printf("x + z: Invalid operation\n");
```

```
}
result = checkBinaryOperation("y", "arr", '*');
if (result != TYPE_ERROR) {
printf("y * arr: Valid operation, result type: %s\n",
result == TYPE_FLOAT ? "FLOAT" : "INT");
}
return 0;
}
```

output:

```
kits@kits-Veriton-M200-H610:~/Desktop$ cc typecheck.c
kits@kits-Veriton-M200-H610:~/Desktop$ ./a.out
Type Checking System
==================

Added variable x: int
Added variable y: float
Added variable z: char
Added variable arr: int array

Type checking operations:
x + y: Valid operation, result type: FLOAT
Error: Type mismatch in operation x + z
x + z: Invalid operation
y * arr: Valid operation, result type: FLOAT
```